

Airport Management Information System Report

Airport Management Information System

CMPE343 DBMS-I



Table of Contents

1. Introduction
2. Project Objectives
3. Database Design
 - 3.1 Main Entities
 - 3.2 ER Diagram
 - 3.3 Relational Schema
4. Creating Tables & Relations
5. Populating Data
6. Displaying Tables
7. SQL Queries and Outputs
8. Conclusion

1. Introduction

The Airport Management Information System is a database project developed to improve the organization and management of airport operations. The system is designed to store, manage, and retrieve airport-related information efficiently and accurately.

The project focuses on managing different airport components such as airlines, airplanes, airplane models, hangars, employees, technicians, traffic controllers, tests, testing events, and flights. The database reduces data redundancy, improves consistency, and provides easy access to operational records.

The system is implemented using MySQL and follows database design principles including Entity Relationship Diagrams (ERD), relational schemas, primary keys, foreign keys, and normalization techniques.

Additionally, the project allows airport administrators to monitor airplane testing history, employee records, technician expertise, hangar allocation, and flight schedules. The database structure is flexible and supports future expansion of airport services.

This project was developed as part of the Database Management Systems course to demonstrate practical database design and SQL implementation skills.

2. Project Objectives

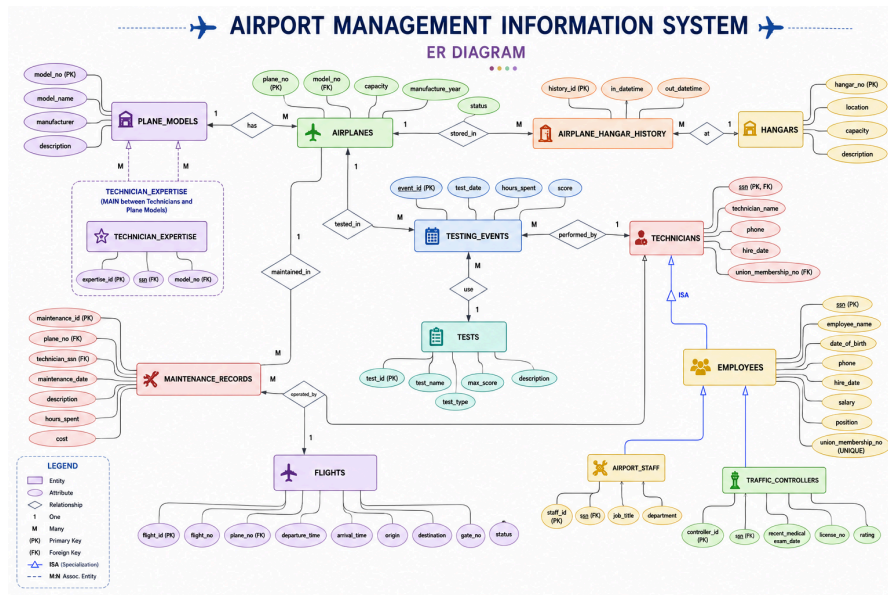
- Store airplane information efficiently
- Manage airline records
- Track flights and testing events
- Manage employee information
- Monitor hangar allocations
- Reduce data redundancy
- Improve airport organization
- Support efficient airport operations

3. Database Design

3.1 Main Entities

AIRPLANES  Stores airplane information	TECHNICIANS  Stores technician records	FLIGHTS  Stores flight information	PLANE MODELS  Stores airplane model details
EMPLOYEES  Stores employee records	TESTS  Stores airplane tests	HANGARS  Stores hangar information	MAINTENANCE  Stores maintenance records

3.2 ER Diagram



The ER Diagram illustrates the relationships between entities in the Airport Management Information System database.

Relationships:

- One Airline can own multiple Airplanes
- One Plane Model can belong to multiple Airplanes
- One Hangar can contain multiple Airplanes
- One Employee can be either a Technician or Traffic Controller
- One Technician can perform multiple Tests
- One Airplane can have multiple Test Events
- One Airplane can perform multiple Flights

3.3 Relational Schema

Airline

- airline_id (PK)
- airline_name
- country

Plane_Model

- model_id (PK)
- model_name
- manufacturer
- max_range

Airplane

- plane_id (PK)
- plane_no
- model_id (FK)
- airline_id (FK)
- capacity
- manufacture_year
- status

Hangar

- hangar_id (PK)
- hangar_no
- location
- capacity

Employee

- ssn (PK)
- first_name
- last_name
- phone
- salary
- union_membership_no
- hire_date

Technician

- technician_id (PK)
- ssn (FK)
- specialization_level

Traffic_Controller

- controller_id (PK)
- ssn (FK)

- recent_medical_exam

Test

- test_id (PK)
- test_name
- max_score
- frequency_days

Test_Event

- event_id (PK)
- plane_id (FK)
- technician_id (FK)
- test_id (FK)
- test_date
- hours_spent
- score

Flight

- flight_id (PK)
- plane_id (FK)
- departure_airport
- destination_airport
- departure_time
- arrival_time

4. Creating Tables & Relations

1. Airline

```
CREATE TABLE Airline (
    airline_id INT PRIMARY KEY AUTO_INCREMENT,
    airline_name VARCHAR(100) NOT NULL UNIQUE,
    country VARCHAR(100) NOT NULL
);
```

2. Plane_Model:

```
CREATE TABLE Plane_Model (
```

```
model_id INT PRIMARY KEY AUTO_INCREMENT,  
model_name VARCHAR(100) NOT NULL,  
manufacturer VARCHAR(100) NOT NULL,  
max_range INT CHECK (max_range > 0)  
);
```

3.Airplane:

```
CREATE TABLE Airplane (  
    plane_id INT PRIMARY KEY AUTO_INCREMENT,  
    plane_no VARCHAR(20) NOT NULL UNIQUE,  
    model_id INT NOT NULL,  
    airline_id INT NOT NULL,  
    capacity INT CHECK (capacity > 0),  
    manufacture_year YEAR,  
    status VARCHAR(50) DEFAULT 'Active',  
    FOREIGN KEY (model_id)  
        REFERENCES Plane_Model(model_id),  
    FOREIGN KEY (airline_id)  
        REFERENCES Airline(airline_id)  
);
```

4..Hanger:

```
.CREATE TABLE Hangar (  
    hangar_id INT PRIMARY KEY AUTO_INCREMENT,  
    hangar_no VARCHAR(20) NOT NULL UNIQUE,  
    location VARCHAR(100),
```



```
capacity INT CHECK (capacity > 0)
);

5.Airplane_Hangar_History:

CREATE TABLE Airplane_Hangar_History (
    history_id INT PRIMARY KEY AUTO_INCREMENT,
    plane_id INT NOT NULL,
    hangar_id INT NOT NULL,
    in_datetime DATETIME NOT NULL,
    out_datetime DATETIME,
    FOREIGN KEY (plane_id)
        REFERENCES Airplane(plane_id),

    FOREIGN KEY (hangar_id)
        REFERENCES Hangar(hangar_id)
);
```

```
6.Employee:

CREATE TABLE Employee (
    ssn VARCHAR(20) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone VARCHAR(20),
    salary DECIMAL(10,2),
    union_membership_no VARCHAR(50) UNIQUE,
    hire_date DATE
```

);

7. Technician:

```
CREATE TABLE Technician (  
    technician_id INT PRIMARY KEY AUTO_INCREMENT,  
    ssn VARCHAR(20) UNIQUE,  
    specialization_level VARCHAR(50),  
    FOREIGN KEY (ssn)  
        REFERENCES Employee(ssn)  
);
```

8. Traffic_Controller :

```
CREATE TABLE Traffic_Controller (  
    controller_id INT PRIMARY KEY AUTO_INCREMENT,  
    ssn VARCHAR(20) UNIQUE,  
    recent_medical_exam DATE,  
    FOREIGN KEY (ssn)  
        REFERENCES Employee(ssn)  
);
```

9. Test:

```
CREATE TABLE Test (  
    test_id INT PRIMARY KEY AUTO_INCREMENT,  
    test_name VARCHAR(100) NOT NULL,  
    max_score INT,  
    frequency_days INT  
);
```

10. Test_Event:

```
CREATE TABLE Test_Event (  
    event_id INT PRIMARY KEY AUTO_INCREMENT,  
    plane_id INT NOT NULL,  
    technician_id INT NOT NULL,  
    test_id INT NOT NULL,  
    test_date DATE,  
    hours_spent DECIMAL(5,2),  
    score INT,  
    FOREIGN KEY (plane_id)  
        REFERENCES Airplane(plane_id),  
  
    FOREIGN KEY (technician_id)  
        REFERENCES Technician(technician_id),  
  
    FOREIGN KEY (test_id)  
        REFERENCES Test(test_id)  
);
```

11. Technician_Expertise:

```
CREATE TABLE Technician_Expertise (  
    technician_id INT,  
    model_id INT,  
    PRIMARY KEY (technician_id, model_id),  
    FOREIGN KEY (technician_id)  
        REFERENCES Technician(technician_id),
```

```

FOREIGN KEY (model_id)

REFERENCES Plane_Model(model_id)

);

12.Flight:

CREATE TABLE Flight (

    flight_id INT PRIMARY KEY AUTO_INCREMENT,

    plane_id INT,

    departure_airport VARCHAR(100),

    destination_airport VARCHAR(100),

    departure_time DATETIME,

    arrival_time DATETIME,

    FOREIGN KEY (plane_id)

        REFERENCES Airplane(plane_id)

);

```

5. Populating Data

1.Airline:

```

INSERT INTO Airline (airline_name, country)
VALUES
('Turkish Airlines', 'Turkey'),
('Pegasus Airlines', 'Turkey'),
('Qatar Airways', 'Qatar'),
('Lufthansa', 'Germany');

```

2 Plane_Model:

```

INSERT INTO Plane_Model
(model_name, manufacturer, max_range)
VALUES
('Boeing 737', 'Boeing', 5600),
('Airbus A320', 'Airbus', 6100),

```

```
('Boeing 777', 'Boeing', 15600),  
( 'Airbus A350', 'Airbus', 15000);
```

3 Airplane:

```
INSERT INTO Airplane  
(plane_no, model_id, airline_id,  
 capacity, manufacture_year, status)  
VALUES  
( 'TC-100', 1, 1, 180, 2018, 'Active'),  
( 'TC-101', 2, 2, 200, 2020, 'Active'),  
( 'QT-500', 3, 3, 350, 2019, 'Maintenance'),  
( 'LH-777', 4, 4, 320, 2021, 'Active');
```

4 Hangar:

```
INSERT INTO Hangar  
(hangar_no, location, capacity)  
VALUES  
( 'H1', 'North Zone', 5),  
( 'H2', 'South Zone', 7),  
( 'H3', 'East Zone', 4);
```

5 Airplane_Hangar_History:

```
INSERT INTO Airplane_Hangar_History  
(plane_id, hangar_id, in_datetime, out_datetime)  
VALUES  
(1, 1, '2026-05-01 08:00:00', NULL),  
(2, 2, '2026-05-02 10:00:00', NULL),  
(3, 3, '2026-05-03 12:00:00', '2026-05-05 14:00:00');
```

6 Employee:

```
INSERT INTO Employee  
(ssn, first_name, last_name, phone,  
 salary, union_membership_no, hire_date)  
VALUES  
( '1001', 'Ali', 'Demir', '5551111', 6000, 'UN001', '2020-01-10'),  
( '1002', 'Ayse', 'Kara', '5552222', 7500, 'UN002', '2019-03-15'),  
( '1003', 'John', 'Smith', '5553333', 8000, 'UN003', '2021-06-20'),  
( '1004', 'Maria', 'Lopez', '5554444', 7200, 'UN004', '2018-09-11'),  
( '1005', 'Omar', 'Hassan', '5555555', 6700, 'UN005', '2022-02-01');
```

7 Technician:

```
INSERT INTO Technician  
(ssn, specialization_level)  
VALUES  
( '1001', 'Senior'),
```

```
('1003', 'Intermediate'),  
( '1005', 'Expert');
```

```
8 Traffic_Controller:  
INSERT INTO Traffic_Controller  
(ssn, recent_medical_exam)  
VALUES  
( '1002', '2026-01-10'),  
( '1004', '2025-11-20');
```

```
9 Test:  
INSERT INTO Test  
(test_name, max_score, frequency_days)  
VALUES  
( 'Engine Inspection', 100, 180),  
( 'Safety Inspection', 100, 365),  
( 'Electrical Systems Test', 100, 90),  
( 'Hydraulic Test', 100, 120);
```

```
10 Test_Event :  
INSERT INTO Test_Event  
(plane_id, technician_id, test_id,  
 test_date, hours_spent, score)  
VALUES  
(1, 1, 1, '2026-05-01', 5.5, 92),  
(1, 1, 2, '2026-05-02', 4.0, 88),  
(2, 2, 1, '2026-05-03', 6.0, 95),  
(3, 3, 3, '2026-05-04', 3.5, 90),  
(4, 1, 4, '2026-05-05', 7.0, 97);
```

```
11 Technician_Expertise :  
INSERT INTO Technician_Expertise  
(technician_id, model_id)  
VALUES  
(1, 1),  
(1, 2),  
(2, 3),  
(3, 4);
```

```
12 Flight :  
INSERT INTO Flight  
(plane_id, departure_airport,  
 destination_airport,
```

```

departure_time, arrival_time)
VALUES
(1, 'Ercan Airport', 'Istanbul Airport',
'2026-05-10 08:00:00',
'2026-05-10 10:00:00'),

(2, 'Ercan Airport', 'Doha Airport',
'2026-05-11 09:00:00',
'2026-05-11 13:00:00'),

(3, 'Ercan Airport', 'Berlin Airport',
'2026-05-12 07:00:00',
'2026-05-12 12:00:00');

```

6. Displaying Tables

1.Airline Table

airline_id	airline_name	country
1	Turkish Airlines	Turkey
2	Pegasus Airlines	Turkey
3	Qatar Airways	Qatar
4	Lufthansa	Germany

2.Plane_Model Table

model_id	model_name	manufacturer	max_range
1	Boeing 737	Boeing	5600
2	Airbus A320	Airbus	6100
3	Boeing 777	Boeing	15600
4	Airbus A350	Airbus	15000

3.Airplane Table

plane_id	plane_no	model_id	airline_id	capacity	manufacture_year	status
----------	----------	----------	------------	----------	------------------	--------

1	TC-100	1	1	180	2018	Active
2	TC-101	2	2	200	2020	Active
3	QT-500	3	3	350	2019	Maintenance
4	LH-777	4	4	320	2021	Active

4. Hangar Table

hangar_id	hangar_no	location	capacity
1	H1	North Zone	5
2	H2	South Zone	7
3	H3	East Zone	4

5. HANGAR HISTORY

history_id	plane_id	hangar_id	in_datetime	out_datetime
1	1	1	2026-05-01 08:00:00	NULL
2	2	2	2026-05-02 10:00:00	NULL
3	3	3	2026-05-03 12:00:00	2026-05-05 14:00:00

6. Employee Table

ssn	first_name	last_name	salary
1001	Ali	Demir	6000
1002	Ayşe	Kara	7500
1003	John	Smith	8000
1004	Maria	Lopez	7200

1005	Omar	Hassan	6700
------	------	--------	------

7. TECHNICIAN table

technician_id	ssn	specialization_level
1	1001	Senior
2	1003	Intermediate
3	1005	Expert

8. TRAFFIC CONTROLLER

controller_id	ssn	recent_medical_exam
1	1002	2026-01-10
2	1004	2025-11-20

9. TEST

test_id	test_name	max_score	frequency_days
1	Engine Inspection	100	180
2	Safety Inspection	100	365
3	Electrical Systems Test	100	90
4	Hydraulic Test	100	120

10. TEST EVENT

event_id	plane_id	technician_id	test_id	score
1	1	1	1	92
2	1	1	2	88
3	2	2	1	95

4	3	3	3	90
5	4	1	4	97

11.TECHNICIAN EXPERTISE

technician_id	model_id
1	1
1	2
2	3
3	4

12.FLIGHT

flight_id	plane_id	departure_airport	destination_airport
1	1	Ercan Airport	Istanbul Airport
2	2	Ercan Airport	Doha Airport
3	3	Ercan Airport	Berlin Airport

7. SQL Queries and Outputs

1. Show All Airplanes

```
SELECT * FROM Airplane;
```

Output

plane_no	capacity	status
TC-100	180	Active
TC-101	200	Active
QT-500	350	Mainten ance
LH-777	320	Active

2. Total Number of Airplanes

```
SELECT COUNT(*) AS total_airplanes
FROM Airplane;
```

Output

total_airplanes
4

3. Average Airplane Capacity

```
SELECT AVG(capacity) AS average_capacity
FROM Airplane;
```

Output

average_capacity
262.5

4. Highest Test Score

```
SELECT MAX(score) AS highest_score
FROM Test_Event;
```

Output

highest_score
97

5. Total Testing Hours

```
SELECT SUM(hours_spent) AS total_testing_hours
FROM Test_Event;
```

Output

total_testing_hours
26.00

6. Airplanes Currently in Hangars

```
SELECT a.plane_no, h.hangar_no
FROM Airplane a
JOIN Airplane_Hangar_History ahh
ON a.plane_id = ahh.plane_id
JOIN Hangar h
ON ahh.hangar_id = h.hangar_id
WHERE ahh.out_datetime IS NULL;
```

Output

plane_no	hangar_no
TC-100	H1
TC-101	H2

7. Airplanes with Airline Names

```
SELECT a.plane_no, al.airline_name, a.capacity
FROM Airplane a
JOIN Airline al
ON a.airline_id = al.airline_id;
```

Output

plane_no	airline_name	capacity
TC-100	Turkish Airlines	180
TC-101	Pegasus Airlines	200
QT-500	Qatar Airways	350
LH-777	Lufthansa	320

8. Technicians and Their Expertise

```
SELECT t.technician_id, pm.model_name
FROM Technician_Expertise te
JOIN Technician t
ON te.technician_id = t.technician_id
JOIN Plane_Model pm
ON te.model_id = pm.model_id;
```

Output

technician_id	model_name
1	Boeing 737
1	Airbus A320
2	Boeing 777
3	Airbus A350

9. Controllers Needing Medical Exam

```
SELECT *
FROM Traffic_Controller
WHERE recent_medical_exam <
CURDATE() - INTERVAL 1 YEAR;
```

Output

controller_id	ssn
2	1004

10. Airplanes Under Maintenance

```
SELECT plane_no
FROM Airplane
WHERE status = 'Maintenance';
```

Output

plane_no
QT-500

11. Total Tests Per Technician

```
SELECT technician_id,  
COUNT(*) AS total_tests  
FROM Test_Event  
GROUP BY technician_id;
```

Output

technician_id	total_tests
1	3
2	1
3	1

12. Monthly Test Statistics

```
SELECT MONTH(test_date) AS month,  
COUNT(*) AS total_tests  
FROM Test_Event  
GROUP BY MONTH(test_date);
```

Output

month	total_tests
5	5

13. Average Test Score

```
SELECT AVG(score) AS average_score  
FROM Test_Event;
```

Output

average_score
92.4

14. Employees Hired After 2020

```
SELECT *  
FROM Employee  
WHERE hire_date > '2020-01-01';
```

Output

first_name	last_name	hire_date
Ali	Demir	2020-01-10
John	Smith	2021-06-20
Omar	Hassan	2022-02-01

15. Flight Destinations

```
SELECT departure_airport,  
destination_airport  
FROM Flight;
```

Output

departure_airport	destination_airport
Ercan Airport	Istanbul Airport
Ercan Airport	Doha Airport
Ercan Airport	Berlin Airport

8. Conclusion

The Airport Management Information System demonstrates how relational databases can improve airport operations efficiently and accurately.

The project applies important database concepts such as ER diagrams, relational schemas, SQL queries, normalization techniques, primary keys, and foreign keys to organize airport data in a structured manner.

The system supports airport management activities including airplane monitoring, flight scheduling, employee management, testing operations, and hangar allocation.

Overall, the project successfully demonstrates practical database implementation skills using MySQL and provides a strong foundation for future airport management applications.